

SSAFY

데이터 다루기

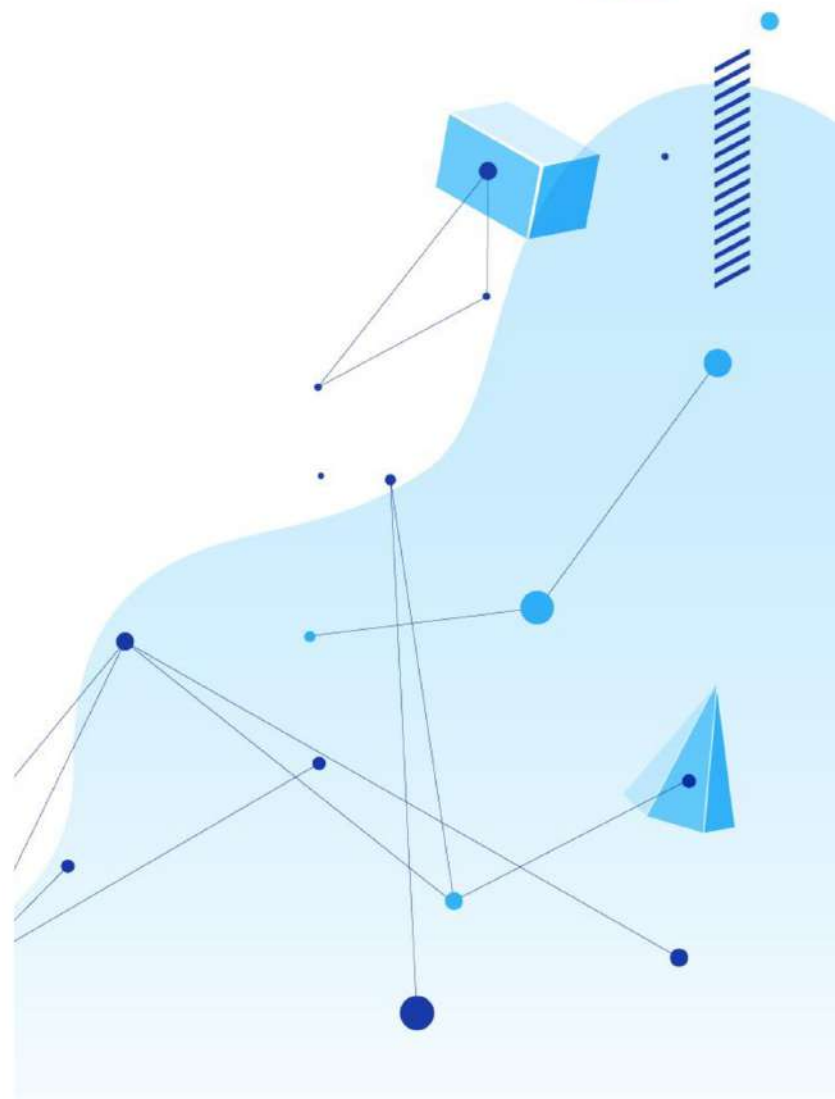


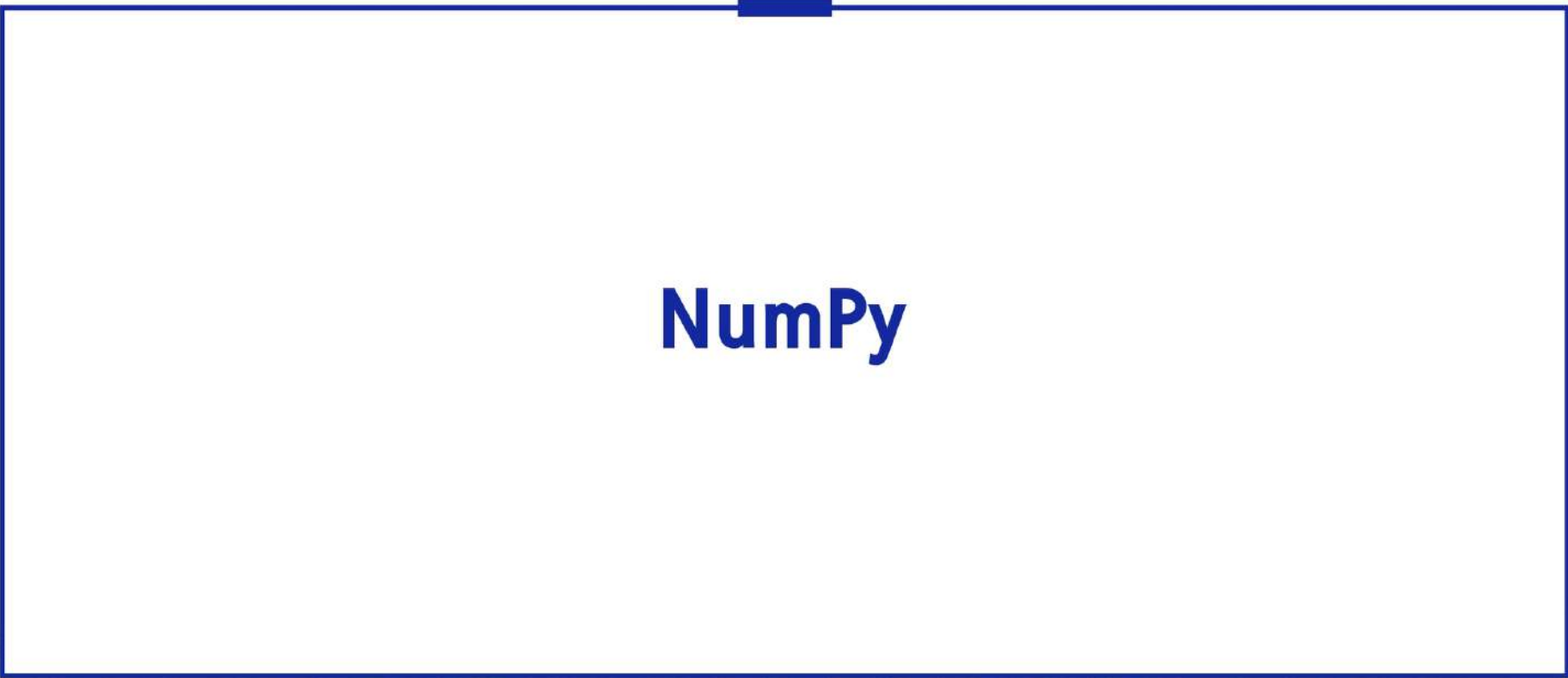
목차

01 NumPy

02 Pandas

03 정리 & 확인 문제





NumPy

엑셀 파일 통합

- 매월 부서별 판매 기록이 담긴 엑셀 파일을 하나로 통합
- 복사, 붙여넣기, 정렬, 합계 재계산을 수작업으로 진행하면 많은 시간 소요
- 데이터가 늘어날수록 반복 작업과 작업 시간도 증가



부서	매출
영업부	6,200
인사부	3,100
개발부	8,400

👉 손으로 복사·정렬

- 간단한 코드로 데이터 정렬과 집계 가능
- 데이터가 늘어나도 같은 코드를 재사용해 작업 가능

```
In [ ]: df.sort_values('매출', ascending=False)
```

```
df.sort_values("매출", ascending=False)
```



부서	매출 ▼
개발부	8,400
영업부	6,200
인사부	3,100

NumPy와 Pandas

※ 자료구조(Data Structure) : 여러 데이터를 목적에 맞는 형태로 저장하고 다루기 위한 구조

- Pandas : 표 형태의 데이터를 다루며 내부적으로 NumPy 배열을 활용
- 배열(ndarray) : 숫자 데이터를 처리하는 NumPy의 다차원 자료구조



NumPy 위에서 Pandas가 동작

리스트와 NumPy 배열

파이썬 리스트 연산

- 리스트의 덧셈은 각 원소를 더하지 않고 두 리스트를 연결
- 리스트에 숫자를 곱하면 각 원소를 계산하지 않고 리스트를 반복
- 각 원소에 대해 계산하려면 반복문을 이용해 값을 하나씩 처리
- 가격 리스트에 10% 할인을 적용하려면 각 가격을 개별적으로 계산해야 함

```
In [ ]: 가격 = [4000, 4500, 5000]
        가격 * 2      # [4000, 4500, 5000, 4000, 4500, 5000]
        # 각 가격에 2를 곱하는 것이 아닌 리스트를 두 번 반복
```

가격 = [4000, 4500, 5000] 가격 * 2



[4000, 4500, 5000, 4000, 4500, 5000] 길이가 두 배

NumPy 배열의 특징

- 배열(ndarray) : 같은 자료형의 데이터를 담는 다차원 자료구조
- 벡터화(Vectorization) : 반복문을 직접 작성하지 않고 배열 전체에 연산을 적용
- 연산 속도 : 벡터화된 연산을 이용하면 일반적인 반복문보다 빠르게 처리 가능
- 집계 함수 : `mean`, `sum`, `max`, `min`, `std` 등을 이용해 배열의 값을 계산

```
In [ ]: import numpy as np
        가격 = np.array([4000, 4500, 5000])
        가격 * 0.9      # array([3600., 4050., 4500.]) ← 원소별 10% 할인
        print(가격.dtype) # int64 ← 원소 자료형
```

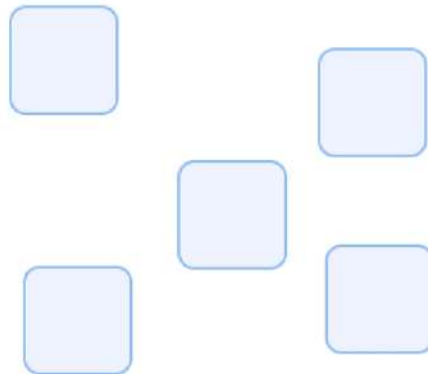

NumPy 배열의 연산 속도

※ 참조(Reference) : 값 자체가 아니라 객체가 저장된 메모리 위치를 가리키는 정보

※ 객체(Object) : 파이썬에서 값과 자료형, 관련 기능을 묶어 다루는 단위

- 리스트 : 각 객체의 참조를 저장하며 반복문으로 값을 하나씩 처리
- NumPy 배열 : 같은 자료형의 데이터를 연속된 메모리 공간에 저장하며 배열 전체를 벡터화 연산으로 처리

흩어진 객체 · 리스트



연속 메모리 · 배열



파이썬과 NumPy 성능 비교

- 같은 범위의 숫자를 더하는 파이썬 합계 연산과 NumPy 합계 연산의 실행 시간을 비교
- 이 측정에서는 데이터가 많아질수록 NumPy 연산의 속도 차이가 커짐
- 실행 시간과 속도 차이는 실행 환경에 따라 달라질 수 있음

	파이썬(초)	NumPy(초)	속도 차이
1,000	0.0000154	0.0000053	2.9배
100,000	0.0013191	0.0000637	20.7배
1,000,000	0.0109592	0.0003693	29.7배

```
In [ ]: import numpy as np

arr = np.arange(1_000_000)
arr_sum = arr.sum()           # NumPy 배열의 합계 계산
list_sum = sum(range(1_000_000)) # 같은 범위의 합계 계산
print(arr_sum, list_sum)      # 4999999500000 4999999500000, 결과
```

NumPy 배열 생성

배열 만들기

- `np.array(리스트)` : 리스트를 NumPy 배열로 변환
- `np.arange(n)` : 0부터 n-1까지의 정수로 배열을 생성
- `np.zeros(n)`, `np.ones(n)` : 0 또는 1로 채운 배열을 생성
- `np.linspace(a, b, n)` : a부터 b까지 같은 간격의 값 n개를 생성
- `.shape` : 배열의 각 차원 크기를 확인

```
In [ ]: import numpy as np

수량 = np.array([2, 1, 3, 5, 1])
print(수량)           # [2 1 3 5 1]
print(수량.shape)     # (5,), 원소 5개인 1차원 배열
print(np.arange(5))   # [0 1 2 3 4]
print(np.zeros(3))    # [0. 0. 0.]
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.]
```

벡터 연산

※ 벡터(Vector) : 여러 숫자를 한 줄로 나열한 1차원 배열

- 배열과 숫자의 연산은 배열의 모든 원소에 적용
- 크기가 같은 배열끼리 연산하면 같은 위치의 원소를 계산
- 반복문을 작성하지 않고 배열 전체를 처리

```
In [ ]: 단가 = np.array([4000, 4500, 5000, 5000, 6000])
        수량 = np.array([2, 1, 3, 5, 1])

        매출 = 단가 * 수량      # 같은 위치의 단가와 수량을 곱해 매출 계산
        print(매출)             # [ 8000  4500 15000 25000 6000]
        print(단가 * 0.9)       # 각 단가에 10% 할인 적용
```

단가						수량						매출				
4000	4500	5000	5000	6000	×	2	1	3	5	1	=	8000	4500	15000	25000	6000

2차원 배열

2차원 배열 만들기

- `np.array([[1, 2, 3], [4, 5, 6]])` : 2차원 배열을 생성
- `arr.reshape(행, 열)` : 1차원 배열을 지정한 행과 열의 2차원 배열로 변환
- `arr.T` : 배열의 행과 열을 전환
- `.shape` : 배열의 행과 열 수를 반환

```
In [ ]: import numpy as np

매출표 = np.array([
    [8000, 4500, 15000], # 서울
    [6000, 5200, 10000]  # 부산
])

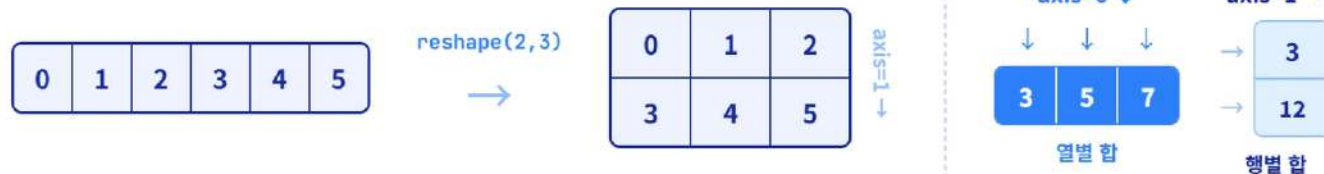
print(매출표.shape)      # (2, 3), 2행 3열
print(매출표[0, 1])      # 4500, 0행 1열의 값
print(매출표.T)          # 행과 열을 전환한 3행 2열 배열
```

reshape와 축 연산

- **reshape(행, 열)** : 원소 수를 유지하면서 배열의 형태를 변경
- **-1** : 전체 원소 수에 맞춰 해당 차원의 크기를 계산
- **axis=0** : 열별로 값을 집계
- **axis=1** : 행별로 값을 집계

```
In [ ]: arr = np.arange(6)           # [0 1 2 3 4 5]
        ㄱ = arr.reshape(2, 3)       # [[0 1 2]
                                     #  [3 4 5]]
        print(arr.reshape(3, -1))    # 3행 2열, 열 크기를 계산

        print(ㄱ.sum(axis=0))        # [3 5 7], 열별 합계
        print(ㄱ.sum(axis=1))        # [3 12], 행별 합계
```



NumPy 집계 함수

※ 표준편차(Standard Deviation) : 데이터가 평균을 기준으로 얼마나 퍼져 있는지 나타내는 값

- `.mean()` : 평균
- `.sum()` : 합계
- `.max()` : 최댓값
- `.min()` : 최솟값
- `.std()` : 표준편차
- 배열 메서드인 `arr.mean()` 또는 NumPy 함수인 `np.mean(arr)` 형식으로 사용 가능

```
In [ ]: 매출 = np.array([8000, 4500, 15000, 25000, 6000])
print(매출.sum())      # 58500 ← 총매출
print(매출.mean())     # 11700.0 ← 평균 매출
print(매출.max())      # 25000 ← 최고 매출
```



브로드캐스팅(Broadcasting)

※ 스칼라(Scalar) : 배열이 아닌 하나의 숫자 값

※ `np.newaxis` : 배열에 새로운 축을 추가하는 표기로, (3,) 배열을 (3, 1)처럼 확장할 때 사용

브로드캐스팅 : 크기가 다른 배열을 연산할 수 있도록 작은 배열을 확장하는 방식

- 1차원 배열과 스칼라 : 스칼라를 배열의 모든 원소에 적용
- 크기가 같은 1차원 배열 : 같은 위치의 원소끼리 연산
- 1차원 배열과 2차원 배열 : 크기가 연산 조건에 맞으면 1차원 배열을 축에 따라 확장해 연산

```
In [ ]: 단가 = np.array([4000, 5000, 6000])      # (3,)
        수량 = np.array([[1, 2], [3, 1], [5, 2]]) # (3, 2)

        결과 = 단가[:, np.newaxis] * 수량      # (3,1) * (3,2) → (3,2)
        print(결과)
        # [[ 4000  8000]
        #   [15000  5000]
        #   [30000 12000]]
```


배열 곱셈 결과 비교

- 질문: `np.array([1,2,3]) * 2`의 결과는?

배열 곱셈 결과는?

`arr * 2`

A

`[1, 2, 3, 1, 2, 3]`

B

`[2, 4, 6]`

배열 곱셈 결과 비교

• 질문: `np.array([1,2,3]) * 2`의 결과는?

배열 곱셈 결과는?

`arr * 2`

A

`[1, 2, 3, 1, 2, 3]`

B · 정답

`[2, 4, 6]`



배열은 모든 원소에 연산이 한 번에 적용 → `[2, 4, 6]`

조건부 필터링 (boolean indexing)

- **배열[조건]** : 조건이 True인 위치의 원소만 추출
- **&** : 두 조건을 모두 만족하는 원소를 선택
- **|** : 두 조건 중 하나 이상을 만족하는 원소를 선택
- 여러 조건을 결합할 때에는 각 조건을 괄호로 구분

```
In [ ]: 매출 = np.array([8000, 4500, 15000, 25000, 6000])

        high = 매출[매출 >= 10000]
        print(high)    # [15000 25000]

        mid = 매출[(매출 >= 5000) & (매출 <= 15000)]
        print(mid)     # [ 8000 15000  6000]
```

- `np.sort(배열)`: 원본을 유지하고 오름차순으로 정렬한 배열을 반환
- `배열[::-1]`: 배열의 순서를 반대로 변경
- `np.argsort(배열)`: 배열을 오름차순으로 정렬할 인덱스 순서를 반환

```
In [ ]: 매출 = np.array([8000, 4500, 15000, 25000, 6000])

print(np.sort(매출))      # [ 4500  6000  8000 15000 25000], 오름차순
print(np.sort(매출)[::-1]) # [25000 15000  8000  6000  4500], 내림차순
print(np.argsort(매출))   # [1 4 0 2 3], 오름차순 정렬 기준 인덱스
```

Pandas

DataFrame과 Series

※ *merge* : 두 표를 공통된 컬럼이나 인덱스를 기준으로 결합하는 기능

- **DataFrame** : 행과 열로 구성된 2차원 표
- **Series** : DataFrame의 한 열을 구성하는 1차원 자료구조
- **DataFrame은 열마다 다른 자료형을 사용할 수 있음**
 - 지역 열은 문자열, 단가 열은 정수로 구성 가능
 - NumPy 배열은 하나의 배열에 같은 자료형을 사용
- **엑셀과 비교하기**
 - 엑셀 시트 = DataFrame
 - 엑셀 행/열 = DataFrame의 행/열
 - 엑셀 VLOOKUP = **merge**
 - 엑셀 피벗 테이블 = **groupby**

엑셀 시트			DataFrame		
	A	B		음료	매출
1	음료	매출	↔ 같은 구조	0	아메리카노 12000
2	라떼	9000		1	라떼 9000

DataFrame의 구조

- DataFrame : 같은 인덱스를 공유하는 여러 Series로 구성
- 인덱스(index) : 각 행을 구분하는 이름
 - 기본값은 0부터 시작하는 정수
- 컬럼(columns) : 열 이름의 목록
- `df.values` : DataFrame의 값을 NumPy 배열로 반환

```
In [ ]: import pandas as pd

df = pd.DataFrame({
    '메뉴': ['아메리카노', '카페라떼'],
    '단가': [4000, 5000]
})
print(df.index)      # RangeIndex(start=0, stop=2, step=1)
print(df.columns)    # Index(['메뉴', '단가'], dtype='object')
print(type(df['단가'])) # <class 'pandas.core.series.Series'>
```


DataFrame으로 하는 일

- 조회 : `head()`, `info()`로 일부 데이터와 자료형을 확인
- 필터 : `df[조건]`으로 조건을 만족하는 행을 선택
- 집계 : `groupby()`로 데이터를 그룹별로 묶어 요약
- 분석 과정 : 파일 불러오기, 조회, 필터, 집계, 시각화 순서로 진행



조회 메서드

※ 결측값(Missing Value) : 값이 비어 있거나 누락된 데이터

※ 요약 통계(Summary Statistics) : 데이터의 개수, 평균, 표준편차, 최솟값, 최댓값 등 분포를 요약한 통계값

메서드, 속성	반환값	용도	예시
<code>head(n)</code>	DataFrame	위에서 n개 행을 확인	<code>df.head(10)</code>
<code>tail(n)</code>	DataFrame	아래에서 n개 행을 확인	<code>df.tail(5)</code>
<code>info()</code>	콘솔 출력	컬럼, 자료형, 결측값을 확인	<code>df.info()</code>
<code>describe()</code>	DataFrame	수치형 데이터의 요약 통계를 확인	<code>df.describe()</code>
<code>shape</code>	튜플	행과 열의 수를 확인	<code>df.shape</code>
<code>dtypes</code>	Series	각 컬럼의 자료형을 확인	<code>df.dtypes</code>
<code>columns</code>	Index	컬럼 이름을 확인	<code>df.columns</code>

카페 음료 판매 데이터 만들기

- 컬럼 : 지역, 메뉴, 단가, 수량, 매출
- 데이터 수 : 200행

컬럼	의미	예시
지역	판매 지역	서울, 부산, 대전, 광주
메뉴	음료 이름	아메리카노, 카페라떼 등
단가	음료 한 잔의 가격	4,000-6,000
수량	주문 수량	1-5
매출	단가와 수량을 곱한 값	자동 계산

더미데이터 생성 코드

- ※ 더미 데이터(Dummy Data) : 실습이나 테스트를 위해 임의로 만든 데이터
- ※ 난수(Random Number) : 정해진 범위나 규칙에 따라 무작위로 생성한 수

- `np.random.seed(42)`: 난수 값을 고정해 같은 데이터를 재현
- `np.random.choice()`: 지정한 목록에서 지역과 메뉴를 임의로 선택
- 매출 : 단가와 수량을 곱해 계산

```
In [ ]: import numpy as np
import pandas as pd

np.random.seed(42)
n = 200
지역 = np.random.choice(['서울', '부산', '대전', '광주'], n)
메뉴 = np.random.choice(['아메리카노', '카페라떼', '바닐라라떼'], n)
단가 = np.random.choice([4000, 4500, 5000, 5500, 6000], n)
수량 = np.random.randint(1, 6, n)

df = pd.DataFrame({'지역': 지역, '메뉴': 메뉴, '단가': 단가, '수량': 수량})
df['매출'] = df['단가'] * df['수량']
print(df.shape)    # (200, 5)
```

파일 불러오기

pd.read_csv(): CSV 파일을 DataFrame으로 불러오기

- pd.read_csv('경로'): 파일을 표(DataFrame)로 읽어옴
- encoding: 한글이 깨지면 'utf-8-sig' 또는 'cp949'로 지정함
- df.to_csv('경로', index=False): DataFrame을 파일로 저장함

```
In [ ]:
import pandas as pd

df = pd.read_csv('sales.csv')    # 파일 → DataFrame
df.head()                       # 잘 읽혔는지 확인

df.to_csv('result.csv', index=False) # 저장 (인덱스 제외)
```

※ CSV(Comma-Separated Values) : 값을 쉼표로 구분해 표 형태의 데이터를 저장하는 텍스트 파일 형식

※ 인코딩(Encoding) : 문자를 컴퓨터가 저장하고 읽을 수 있는 값으로 변환하는 방식

※ cp949, utf-8-sig : 한글을 저장하고 읽을 수 있는 문자 인코딩 방식으로, utf-8-sig는 유니코드 기반이며 cp949는 주로 윈도우 한글 환경에서 사용

DataFrame 정보 확인

- `df.head(n)`: 위에서 n개 행을 반환하며, n을 생략하면 5개 행을 반환
- `df.info()`: 컬럼, 행 수, 자료형, 결측값 정보를 콘솔에 출력
- `df.shape`: 행과 열의 수를 튜플로 반환
- `df.describe()`: 숫자 컬럼의 요약 통계를 반환

```
In [ ]: df.head()           # 위 5줄 미리보기
        df.info()          # 컬럼, 자료형, 결측 요약
        df.shape           # (200, 5)
        df.describe()      # 숫자 컬럼 통계 (count, mean, std, min, max)
        df['지역'].value_counts() # 지역별 행 수 (빈도)
```

`df.head()`

```
음료 지역 수량 매출
0 아메리카노 강남 3 12000
1 라떼 홍대 2 9000
```

`df.info()`

```
RangeIndex: 100 entries
음료 100 non-null object
매출 100 non-null int64
```

컬럼 선택과 조건 필터

- `df["컬럼"]` : 하나의 컬럼을 Series로 반환
- `df[조건]` : 조건이 True인 행만 반환
- `&` : 두 조건을 모두 만족하는 행을 선택
- `|` : 두 조건 중 하나 이상을 만족하는 행을 선택
- 여러 조건을 결합할 때에는 각 조건을 괄호로 구분

```
In [ ]: # 서울 지역만
        df[df['지역'] == '서울']

        # 매출 1만원 이상이면서 아메리카노만
        df[(df['매출'] >= 10000) & (df['메뉴'] == '아메리카노')]
```

아메리카노	12000
카푸치노	5000
콜드브루	16000
에스프레소	7000

매출 >= 10000



아메리카노	12000
콜드브루	16000

자주 하는 실수

※ 연산자 우선순위(Operator Precedence) : 여러 연산자를 함께 사용했을 때 어떤 연산을 먼저 수행할지 정한 규칙

- **and, or** 대신 **&, |**를 사용
- 각 조건은 괄호로 구분: **(df["매출"] >= 10000)**
 - **==** : 두 값이 같은지 비교
 - **=** : 변수에 값을 대입
- 조건 필터에서는 비교 연산자인 **==**를 사용
- 한글 컬럼도 **df["지역"]** 형식으로 접근
- 괄호를 생략하면 연산자 우선순위로 인해 오류가 발생하거나 다른 결과가 나올 수 있음

```
In [ ]: # 괄호 누락 (예러)
df[df['매출'] >= 10000 & df['메뉴'] == '아메리카노'] # TypeError

# 올바른 작성
df[(df['매출'] >= 10000) & (df['메뉴'] == '아메리카노')]
```



```
df[(df.수량>1) and (df.매출>9000)]
```

ValueError: ambiguous



```
df[(df.수량>1) & (df.매출>9000)]
```

정상 결과 ✓

다중 값 필터와 문자열 검색

- `.isin([값1, 값2])`: 목록에 포함된 값을 가진 행을 선택
- `.str.contains("문자")`: 컬럼 값에 특정 문자가 포함되어 있는지 확인
- `.str.startswith("문자")`: 컬럼 값이 특정 문자로 시작하는지 확인

```
In [ ]: # 아메리카노 또는 카페라떼만
        df[df['메뉴'].isin(['아메리카노', '카페라떼'])]

        # 메뉴명에 '라떼' 포함
        df[df['메뉴'].str.contains('라떼')]

        # 서울이면서 매출 1만원 이상
        df[(df['지역'] == '서울') & (df['매출'] >= 10000)]
```

```
df[df['메뉴'].isin(['아메리카노', '카페라떼'])]
```



2개 메뉴

```
df[df['메뉴'].str.contains('라떼')]
```



'라떼' 행

```
df[(df['지역'] == '서울') & (df['매출'] >= 10000)]
```



서울 고매출

상위, 하위 행 추출

- `.head(n)` : 위에서 n개 행을 반환
- `.sample(n)` : 무작위로 n개 행을 반환
- `.nlargest(n, "컬럼")` : 지정한 컬럼을 기준으로 값이 큰 n개 행을 반환
- `.nsmallest(n, "컬럼")` : 지정한 컬럼을 기준으로 값이 작은 n개 행을 반환

```
In [ ]: # 매출 상위 5개 주문
        df.nlargest(5, '매출')

        # 수량 하위 3개 주문
        df.nsmallest(3, '수량')

        # 무작위 표본 10개
        df.sample(10, random_state=42)
```

loc 라벨 인덱싱

.loc[행 조건, 열 이름] : 행의 조건과 열 이름으로 데이터를 선택

- **df[조건]** : 조건에 맞는 행을 선택
- **.loc** : 행과 열을 함께 선택하거나 값을 변경할 때 사용
- **.iloc[행 위치, 열 위치]** : 정수 위치로 행과 열을 선택

```
In [ ]: # 서울 지역의 매출만 조회
df.loc[df['지역'] == '서울', '매출']

# 조건에 맞는 값 대입 (조건부 대입)
df.loc[df['단가'] < 4000, '단가'] = 4000

df.iloc[0, 1] # 0행 1열 (위치 기반)
```

※ 라벨(Label) : 인덱스나 컬럼 이름처럼 행과 열을 구분하는 이름

groupby 집계

- `groupby("컬럼")` : 같은 값을 가진 행을 그룹으로 묶음
- 그룹별로 `sum`, `mean`, `count`, `max` 등의 집계 함수를 적용
- 엑셀의 피벗 테이블과 비슷한 방식으로 데이터를 요약

```
In [ ]: # 지역별 총매출
        df.groupby('지역')['매출'].sum()

        # 메뉴별 평균 단가
        df.groupby('메뉴')['단가'].mean()

        # 지역별 주문 건수
        df.groupby('지역')['메뉴'].count()
```

다중 groupby와 여러 집계 함수

※ 계층형 인덱스(MultiIndex) : 두 개 이상의 기준으로 구성된 인덱스 구조

- `groupby(["컬럼1", "컬럼2"])` : 두 개 이상의 컬럼을 기준으로 그룹을 생성
- 여러 컬럼으로 그룹을 만들면 계층형 인덱스(MultiIndex)가 생성됨
- `.agg({...})` : 컬럼마다 서로 다른 집계 함수를 적용
- `.reset_index()` : 그룹 기준으로 사용한 인덱스를 일반 컬럼으로 변환

```
In [ ]: # 지역별 + 메뉴별 총매출 (MultiIndex)
df.groupby(['지역', '메뉴'])['매출'].sum()

# 지역별로 매출 합, 평균, 수량 합을 동시에
df.groupby('지역').agg(
    매출합=('매출', 'sum'),
    매출평균=('매출', 'mean'),
    수량합=('수량', 'sum')
)
```


pivot_table

pivot_table() : 행과 열의 두 기준으로 데이터를 교차 집계

- index : 세로축으로 사용할 기준
- columns : 가로축으로 사용할 기준
- values : 집계할 값
- aggfunc : sum, mean과 같은 집계 함수
- groupby()와 비슷하지만 하나의 기준을 가로축으로 배치해 표 형태로 구성

```
In [ ]: # 지역 x 메뉴 별 매출 합계, 평균
df.pivot_table(index='지역',
               columns='메뉴',
               values='매출',
               aggfunc=['sum', 'mean'])
```


groupby 주의사항

※ NaN(Not a Number) : 판다스에서 수치형 데이터의 결측값을 나타낼 때 주로 사용하는 특수한 값

- `["매출"]` : 하나의 집계 컬럼을 선택해 Series로 반환
- `["매출", "수량"]` : 여러 집계 컬럼을 선택해 DataFrame으로 반환
- `count()` : NaN을 제외한 값의 개수를 계산
- `size()` : NaN을 포함한 전체 행 수를 계산
- `mean()` : NaN을 제외하고 평균을 계산
- `inplace=True` : 원본을 변경하므로 필요한 경우 결과를 새로운 변수에 저장

```
In [ ]: df.groupby('지역')['매출'].sum()           # Series 반환
        df.groupby('지역')[['매출', '수량']].sum() # DataFrame 반환

        df.groupby('지역')['메뉴'].count()        # NaN 제외 개수
        df.groupby('지역').size()                 # NaN 포함 전체 행 수
```

Q. "메뉴별 총 판매 수량"을 구하려면?

```
df.groupby('_____')['_____']._____()
```

Q. "메뉴별 총 판매 수량"을 구하려면?

```
df.groupby('메뉴')['수량'].sum()
```

정리 & 확인 문제

1 문제 1

- 다음 중 NumPy 배열의 벡터 연산을 올바르게 설명한 것은?
 - ① 배열 * 2는 배열을 두 번 이어 붙임
 - ② 각 원소를 계산하려면 반복문을 작성해야 함
 - ③ 반복문 없이 배열의 각 원소에 산술 연산을 적용할 수 있음
 - ④ 하나의 배열에 서로 다른 자료형을 자유롭게 저장할 수 있음

1 문제 1

• 다음 중 NumPy 배열의 벡터 연산을 올바르게 설명한 것은?

- ① 배열 * 2는 배열을 두 번 이어 붙임
- ② 각 원소를 계산하려면 반복문을 작성해야 함
- ③ 반복문 없이 배열의 각 원소에 산술 연산을 적용할 수 있음
- ④ 하나의 배열에 서로 다른 자료형을 자유롭게 저장할 수 있음



정답 및 해설

• 정답: ③

• 해설

- ①은 파이썬 리스트의 반복 연산에 해당함
- ②는 벡터 연산을 사용하면 반복문을 작성하지 않아도 되므로 오답
- ③은 배열의 각 원소에 연산을 적용하는 벡터 연산에 해당함
- ④는 NumPy 배열이 같은 자료형의 데이터를 저장하므로 오답

2 문제 2

- Pandas DataFrame에서 부서별 급여의 평균을 구하는 코드는?
 - ① `df['급여'].sum()`
 - ② `df.groupby('부서')['급여'].mean()`
 - ③ `df.sort_values('급여')`
 - ④ `df['부서'].value_counts()`

2 문제 2

- Pandas DataFrame에서 부서별 급여의 평균을 구하는 코드는?
 - ① `df['급여'].sum()`
 - ② `df.groupby('부서')['급여'].mean()`
 - ③ `df.sort_values('급여')`
 - ④ `df['부서'].value_counts()`



정답 및 해설

- 정답: ② `df.groupby("부서")["급여"].mean()`
- 해설
 - ①은 전체 급여의 합계를 계산
 - ②는 부서를 기준으로 그룹화한 후 급여의 평균을 계산
 - ③은 급여를 기준으로 행을 정렬
 - ④는 부서별 행의 개수를 계산

3 문제 3

• DataFrame에서 나이가 30 이상인 행만 선택하는 코드는?

- ① `df[df['나이'] >= 30]`
- ② `df['나이'] >= 30]`
- ③ `df.filter('나이' >= 30)`
- ④ `df.loc[나이 >= 30]`

3 문제 3

- DataFrame에서 나이가 30 이상인 행만 선택하는 코드는?
 - ① `df[df['나이'] >= 30]`
 - ② `df['나이'] >= 30]`
 - ③ `df.filter('나이' >= 30)`
 - ④ `df.loc[나이 >= 30]`



정답 및 해설

- 정답: ① `df[df["나이"] >= 30]`
- 해설
 - ①은 나이 컬럼에 조건을 적용해 조건이 True인 행을 선택
 - ②는 문자열 "나이"와 숫자 30을 비교하므로 오류 발생
 - ③은 `filter()`에 조건식을 전달하는 형식이 아니므로 오답
 - ④는 나이를 컬럼으로 지정하지 않아 오류 발생

핵심 키워드

주제	키워드	핵심
NumPy 배열	<code>np.array()</code> , <code>np.arange()</code> , <code>.shape</code>	배열 생성과 형태 확인
배열 연산	브로드캐스팅, <code>axis=0</code> , <code>axis=1</code>	배열 연산과 축별 계산
배열 형태와 정렬	<code>reshape()</code> , <code>.T</code> , <code>np.sort()</code> , <code>np.argsort()</code>	배열 형태 변경, 전치, 정렬
집계 함수	<code>.mean()</code> , <code>.sum()</code> , <code>.max()</code> , <code>.min()</code> , <code>.std()</code>	배열 값 집계
DataFrame 기초	<code>DataFrame</code> , <code>Series</code> , <code>index</code> , <code>columns</code>	행과 열로 구성된 표 형태의 자료구조
조회와 필터	<code>.head()</code> , <code>.info()</code> , <code>df[조건]</code> , <code>&</code> , <code> </code>	데이터 확인과 조건별 선택
다중 값, 문자열 검색	<code>.isin()</code> , <code>.str.contains()</code> , <code>.str.startswith()</code>	목록과 문자열 조건으로 행 선택
행 추출과 인덱싱	<code>.nlargest()</code> , <code>.nsmallest()</code> , <code>.loc</code> , <code>.iloc</code>	상위, 하위 행 추출과 레이블, 위치 기반 선택
그룹 집계	<code>groupby()</code> , <code>.agg()</code> , <code>pivot_table()</code>	그룹별 요약과 교차 집계

오늘 공부한 내용 요약 및 정리

- NumPy 배열 : 숫자 데이터를 반복문 없이 배열 단위로 계산하며, 리스트보다 빠르게 처리할 수 있음
- `mean()`, `sum()`, `std()` : 배열의 평균, 합계, 표준편차를 계산
- Pandas DataFrame : 데이터를 행과 열로 구성된 표 형태로 다루는 자료구조
- `head()`, `info()`, `describe()` : 표의 구조와 요약 통계를 확인
- 단일 조건과 다중 조건을 이용해 원하는 행을 선택
- `groupby()`에 `sum()`, `mean()`, `count()`를 적용해 그룹별로 집계
- 여러 컬럼을 기준으로 데이터를 그룹화할 수 있음

내일 방송에서 만나요!